

Routing with React Router

In this part, we will build a simple app implementing the following pages:

- A home page that welcomes the user
- A products list page that lists all the products
- A product page that provides details about a particular product
- An admin page for privileged users

This will all be managed using a library called **React Router**.

Through this, we will learn how to implement static links from the products list to the product page and implement route parameters on the product page for the product ID. We will also learn about form navigation and query parameters when it comes to the search feature of our app.

Finally, the part will end with how to lazily load code for a page to improve performance.

So, in this part, we will cover the following topics:

- Introducing React Router
- Declaring routes
- Creating navigation
- Using nested routes
- Using route parameters
- Creating an error page
- Using index routes
- Using search parameters
- Navigating programmatically
- Using form navigation
- Implementing lazy loading

Technical requirements

We will use the following technologies in this part:

- **Browser:** A modern browser such as Google Chrome
- **Node.js** and **npm:** You can install them from <https://nodejs.org/en/download/>
- **Visual Studio Code:** You can install it from <https://code.visualstudio.com/>

Introducing React Router

In this section, we start by creating a new React project for the app before understanding what React Router is and how to install it.

Creating the project

We will develop the app locally using Visual Studio Code, which requires a new Create React App-based project setup. We have covered this several times, so we will not cover the steps in this part – instead, see , *Setting Up React and TypeScript*. Create the project for the app with a name of your choice.

We will style the app with Tailwind CSS. We covered how to install and configure Tailwind in Create React App in previous part , *Approaches to Styling Frontends*, so after you have created the React and TypeScript project for the app, install and configure Tailwind.

Understanding React Router

As the name suggests, React Router is a routing library for React apps. A router is responsible for selecting what to show in the app for a requested path. For example, React Router is responsible for determining what components to render when a path of `/products/6` is requested. For any app containing multiple pages, a router is essential, and React Router has been a popular router library for React for many years.

Installing React Router

React Router is in a package called **react-router-dom**. Install this in the project using the following command in the terminal:

```
npm i react-router-dom
```

TypeScript types are included in **react-router-dom**, so there is no need for a separate installation.

Next, we will create a page in the app and declare a route that shows it.

Declaring routes

We will start this section by creating a page component that lists the app's products. We will then learn how to create a router and declare routes using React Router's **createBrowserRouter** function.

Creating the products list page

The products list page component will contain the list of the React tools in the app. Carry out the following steps to create this:

1. We will start by creating the data source for the page. First, create a folder called **data** in the **src** folder and then a file called **products.ts** within **data**.

2. Add the following content into **products.ts**

```
export type Product = {

  id: number,

  name: string,

  description: string,

  price: number,

};

export const products: Product[] = [

  {

    description:

      'A collection of navigational components that compose declaratively with your application.',

    id: 1,

    name: 'React Router',

    price: 8,
```

```
},
```

```
{
```

```
  description: 'A library that helps manage state      across your app',
```

```
  id: 2,
```

```
  name: 'React Redux',
```

```
  price: 12,
```

```
},
```

```
{
```

```
  description: 'A library that helps you implement    robust forms',
```

```
  id: 3,
```

```
  name: 'React Hook Form',
```

```
  price: 9,
```

```
},
```

```
{  
  
  description: 'A library that helps you interact with      a REST API',  
  
  id: 4,  
  
  name: 'React Apollo',  
  
  price: 10,  
  
},  
  
{  
  
  description: 'A library that provides utility CSS      classes',  
  
  id: 5,  
  
  name: 'Tailwind CSS',  
  
  price: 7,  
  
},  
  
];
```

This is a list of all the React tools in the app held in a JavaScript array.

NOTE

Usually, this kind of data is on a server somewhere, but this adds complexity beyond the scope of this part. We cover how to interact with server data in detail in another part , Interacting with RESTful APIs, including how to do this efficiently with React Router.

3. We will create the products list page component now. First, create a folder for all the page components in the **src** folder called **pages**. Next, create a file called **ProductsPage.tsx** in the **pages** folder for the products list page component.
4. Add the following **import** statement into **ProductsPage.tsx** to import the products we just created:

```
import { products } from '../data/products';
```

5. Next, start to create the **ProductsPage** component by outputting a heading for the page:

```
export function ProductsPage() {
```

```
  return (
```

```
    <div className="text-center p-5">
```

```
      <h2 className="text-xl font-bold text-slate-600">
```

```
        Here are some great tools for React
```

```
      </h2>
```

```
    </div>
```

```
  );
```

```
}
```

This uses Tailwind classes to make the heading large, bold, gray, and horizontally centered.

6. Next, add the list of the products in the JSX:

```
<div className="text-center p-5 text-xl">
```

```
<h2 className="text-base text-slate-600">
```

```
  Here are some great tools for React
```

```
</h2>
```

```
<ul className="list-none m-0 p-0">
```

```
  {products.map((product) => (
```

```
    <li key={product.id} className="p-1 text-base text-slate-800">
```

```
      {product.name}
```

```
    </li>
```

```
  )})
```

```
</ul>
```

```
</div>
```

The Tailwind classes remove the bullet points, margin, and padding from the unordered list element, and make the list items gray.

Notice that we use the `products` array `map` function to iterate over each product and return a `li` element. Using `Array.map` is common practice for JSX looping logic.

Notice the `key` prop on the list item elements. React requires this on elements in a loop to update the corresponding DOM elements efficiently. The value of the `key` prop must be unique and stable within the array, so we have used the product ID.

That completes the product page for now. This page won't show in the app yet because it isn't part of its component tree – we need to declare it as a page using React Router, which we'll do next.

Understanding React Router's router

A router in React Router is a component that tracks the browser's URL and performs navigation. Several routers are available in React Router, and the one recommended for web applications is called a **browser router**. As its name suggests, the `createBrowserRouter` function creates a browser router.

`createBrowserRouter` requires an argument containing all the **routes** in the application. A route contains a path and what component to render when the app's browser address matches that path. The following code snippet creates a router with two routes:

```
const router = createBrowserRouter([
  {
    path: 'some-page',
    element: <SomePage />,
  },
  {
    path: 'another-page',
    element: <AnotherPage />,
  }
]);
```

When the path is `/some-page`, the `SomePage` component will be rendered. When the path is `/another-page`, the `AnotherPage` component will be rendered.

The router returned by `createBrowserRouter` is passed to a `RouterProvider` component and placed high up in the React component

tree, as shown here:

```
const root = ReactDOM.createRoot(  
  document.getElementById('root') as HTMLElement  
);  
root.render(  
  <React.StrictMode>  
    <RouterProvider router={router} />  
  </React.StrictMode>  
);
```

Now that we are starting to understand React Router's router, we will use it in our project.

Declaring the products route

We will declare the products list page in the app using **createBrowserRouter** and **RouterProvider**. Carry out the following steps:

1. We will create our own component to hold all the route definitions.
Create a file called **Routes.tsx** in the **src** folder containing the following **import** statements:

```
import {  
  
  createBrowserRouter,  
  
  RouterProvider,  
  
} from 'react-router-dom';  
  
import { ProductsPage } from '../pages/ProductsPage';
```

We have imported **createBrowserRouter** and **RouterProvider** from React Router. We have also imported **ProductsPage**, which we'll render in a **products** route next.

2. Add the following component underneath the **import** statements to define the router with a **products** route:

```
const router = createBrowserRouter([
```

```

{

    path: 'products',

    element: <ProductsPage />,

},

]);

```

So, the **ProductsPage** component will be rendered when the path is **/products**.

3. Still in **Routes.tsx**, create a component called **Routes** under the router as follows:

```

export function Routes() {

    return <RouterProvider router={router} />;

}

```

This component wraps **RouterProvider** with the router passed into it.

4. Open the **index.tsx** file and add an **import** statement for the **Routes** component we just created beneath the other **import** statements:

```

import { Routes } from './Routes';

```

5. Render **Routes** instead of **App** as the top-level component as follows:

```

root.render(

    <React.StrictMode>

```

```
<Routes />
```

```
</React.StrictMode>
```

```
);
```

This causes the **products** route we defined to be part of the component tree. This means the products list page will be rendered in the app when the path is **/products**.

6. Remove the **import** statement for the **App** component, as this is not needed at the moment.
7. Run the app using **npm start**.

An error screen appears explaining that the current route isn't found:

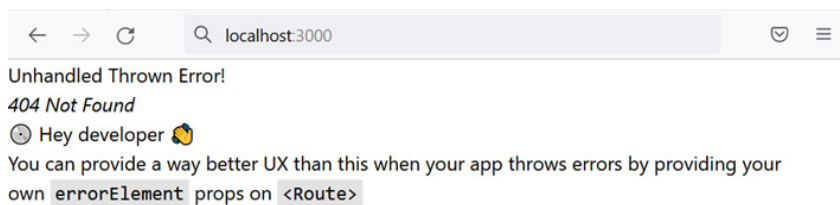


Figure 6.1 – React Router's standard error page

The error page is from React Router. As the error message suggests, we can provide our own error screen, which we will do later in this part.

8. Change the browser URL to **http://localhost:3000/products**.

You will see the products list page component rendered as follows:

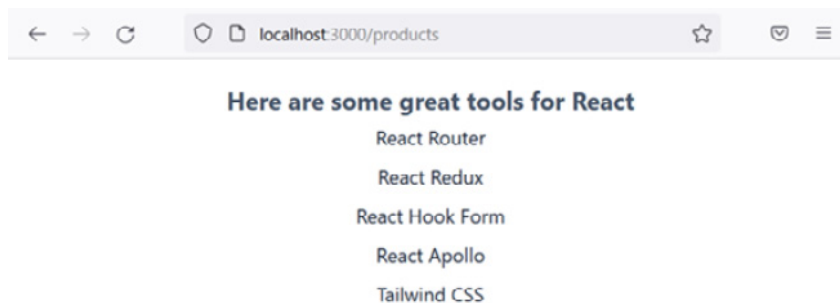


Figure 6.2 – Products list page

This confirms that the **products** route is working nicely. Keep the app running as we recap and move to the next section.

Here's a recap of what we have learned in this section:

- In a web app, routes in React Router are defined using **createBrowserRouter**
- Each route has a path and a component to render when the browser's URL matches that path
- The router returned from **createBrowserRouter** is passed into a **RouterProvider** component, which should be placed high up in the component tree

For more information on **createBrowserRouter**, see the following link in the React Router documentation:

<https://reactrouter.com/en/main/routers/create-browser-router>. For more information on **RouterProvider**, see the following link in the React Router documentation:

<https://reactrouter.com/en/main/routers/router-provider>.

Next, we will learn about React Router components that can perform navigation.

Creating navigation

React Router comes with components called **Link** and **NavLink**, which provide navigation. In this section, we will create a navigation bar at the top of the app containing the **Link** component from React Router. We will then swap **Link** for the **NavLink** component and understand the difference between the two components.

Using the Link component

Carry out the following steps to create an app header containing React Router's **Link** component:

1. Start by creating a file for the app header called **Header.tsx** in the **src** folder containing the following **import** statements:

```
import { Link } from 'react-router-dom';
```

```
import logo from './logo.svg';
```

We have imported the **Link** component from React Router.

We have also imported the React logo because we will include this in the app header with the navigation options.

2. Create the **Header** component as follows:

```
export function Header() {
```

```

return (

  <header className="text-center text-slate-50      bg-slate-900 h-40 p-5">

    <img

      src={logo}

      alt="Logo"

      className="inline-block h-20"

    />

    <h1 className="text-2xl">React Tools</h1>

    <nav></nav>

  </header>

);

}

```

The component contains a **header** element containing the React logo, the app title, and an empty **nav** element. We have used Tailwind classes to make the header gray with the logo and title horizontally centered.

3. Now, create a link inside the **nav** element:

```
<nav>
```

```
<Link
```

```
to="products"
```

```
className="text-white no-underline p-1"
```

```
>
```

```
Products
```

```
</Link>
```

```
</nav>
```

The **Link** component has a **to** prop that defines the path to navigate to. The text to display can be specified in the **Link** content.

4. Open **Routes.tsx** and add an import statement for the **Header** component we just created:

```
import { Header } from './Header';
```

5. In the **router** definition, add a root path that renders the **Header** component as follows:

```
const router = createBrowserRouter([  
  
  {
```

```

    path: '/',

    element: <Header />,

  },

  {

    path: 'products',

    element: <ProductsPage />,

  },

]);

```

What we have just done isn't ideal because the **Header** component needs to show on all routes and not just the root route. However, it will allow us to explore React Router's **Link** component. We will tidy this up in the *Using nested routes* section.

6. In the running app, change the browser address to the root of the app. The new app header appears, containing the **Products** link:

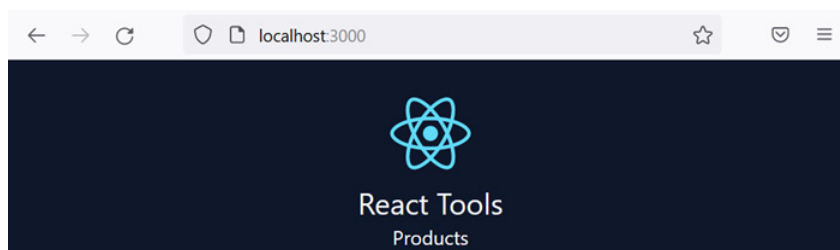


Figure 6.3 – App header

7. Now, inspect the app header elements using the browser's DevTools:

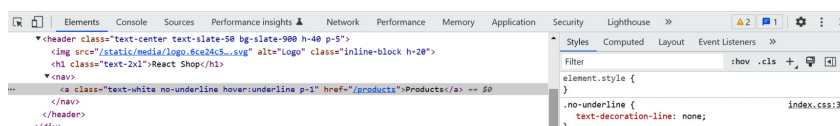


Figure 6.4 – Header component inspection

We can see that the **Link** component is rendered as an HTML anchor element.

8. Select the **Network** tab in DevTools and clear any existing requests that are shown. Click on the **Products** link in the app header. The browser will navigate to the products list page.

Notice that a network request wasn't made for the products list page. This is because React Router overrides the anchor element's default behavior with client-side navigation:

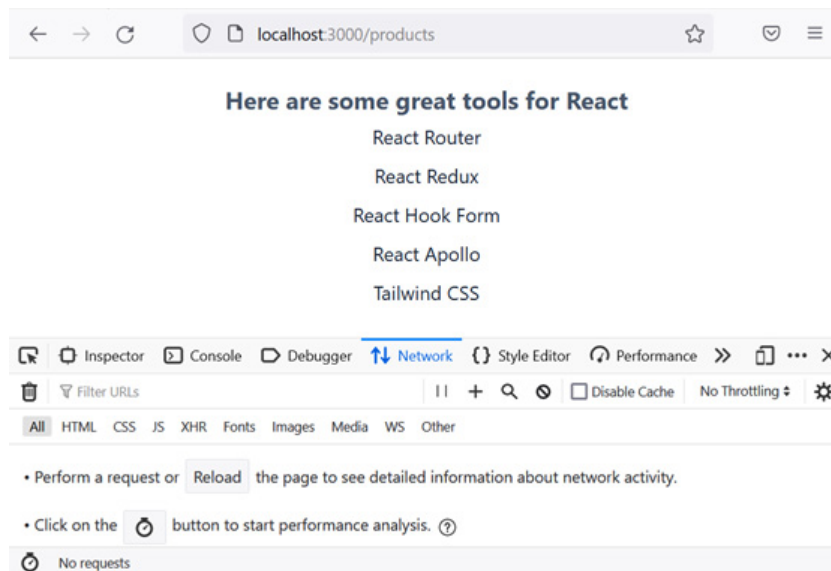


Figure 6.5 – Client-side navigation

Finally, notice that the app header disappears on the products list page, which is not what we want. We will resolve this in the *Using nested routes* section.

Keep the app running as we move to the next section.

The navigation is working well but it would be nice if the **Products** link had a different style when the products list page was active. We will make this improvement next.

Using the NavLink component

React Router's **NavLink** is like a **Link** element but allows it to be styled differently when active. This is really handy for a navigation bar.

Carry out the following steps to replace **Link** with **NavLink** in the app header:

1. Open **Header.tsx** and change the **Link** references to **NavLink**:

```
import { NavLink } from 'react-router-dom';
```

```
...
```

```
export function Header() {
```

```
  return (
```

```
    <header ...>
```

```
      ...
```

```
      <nav>
```

```
        <NavLink
```

```
          to="products"
```

```
          className="..."
```

```
        >
```

```
          Products
```

```
        </NavLink>
```

```
      </nav>
```

```
</header>
```

```
);
```

```
}
```

The app header will look and behave exactly the same at the moment.

2. The **className** prop on the **NavLink** component accepts a function that can be used to conditionally style it, depending on whether its page is active. Update the **className** attribute to the following:

```
<NavLink
```

```
to="products"
```

```
className={({ isActive }) =>
```

```
`text-white no-underline p-1 pb-0.5 border-solid border-b-2 ${
```

```
isActive ? "border-white" : "border-transparent"
```

```
}`
```

```
}
```

```
>
```

```
Products
```

```
</NavLink>
```

The function takes in a parameter, **isActive**, for defining whether the link's page is active. We've added a bottom border to the link if it is active.

We can't see the impact of this change just yet, because the **Products** link doesn't appear on the products list page yet. We will resolve this in the next section.

That completes the app header and our exploration of the **NavLink** component.

To recap, **NavLink** is great for main app navigation when we want to highlight an active link, and **Link** is great for all the other links in our app.

For more information on the **Link** component, see the following link: <https://reactrouter.com/en/main/components/link>. For more information on the **NavLink** component, see the following link: <https://reactrouter.com/en/main/components/nav-link>.

Next, we will learn about nested routes.

Using nested routes

In this section, we will cover **nested routes** and the situations in which they are useful, before using a nested route in our app. The nested route will also resolve the disappearing app header problem we experienced in the previous sections.

Understanding nested routes

A nested route allows a segment of a route to render a component. For example, the following mock-up is commonly implemented using nested routes:

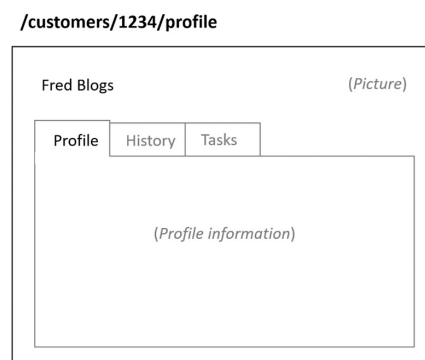


Figure 6.6 – Use case for nested routes

The mock-up displays information about a customer. The path determines the active tab – in the mockup, **Profile** is the active tab because

that is the last segment in the path. If the user selects the **History** tab, the path would change to `/customers/1234/history`.

A **Customer** component could render the shell of this screen, including the customer's name, picture, and tab headings. The component that renders the tab contents could be decoupled from the **Customer** component and coupled to the path instead.

This feature is referred to as *nested routes* because **Route** components are nested inside each other. Here's what the routes for the mock-up could be:

```
const router = createBrowserRouter([
  {
    path: 'customer/:id',
    element: <Customer />,
    children: [
      {
        path: 'profile',
        element: <CustomerProfile />,
      },
      {
        path: 'history',
        element: <CustomerHistory />,
      },
      {
        path: 'tasks',
        element: <CustomerTasks />,
      },
    ],
  },
]);
```

This nested approach to defining routes makes them easy to read and understand, as you can see in the preceding code snippet.

A critical part of nested routes is where child components are rendered in their parent. In the preceding code snippet, where would the **CustomerProfile** component be rendered in the **Customer** component? The solution is React Router's **Outlet** component. Here's an example of **Outlet** in the **Customer** component from the mock-up:

```
export function Customer() {
  ...
  return (
    <div>
      <Name ... />
      <Picture ... />
      <nav>
        <NavLink to="profile" ... >Profile</NavLink>
        <NavLink to="history" ... >History</NavLink>
      </nav>
    </div>
  );
}
```

```

        <NavLink to="tasks" ... >Tasks</NavLink>
      </nav>
      <Outlet />
    </div>
  );
}

```

So, in this example, the **CustomerProfile** component would be rendered after the navigation options in the **Customer** component. Notice that the **Customer** component is decoupled from the nested content. This means new tabs can be added to the customer page without changing the **Customer** component. This is another benefit of nested routes.

Next, we will use a nested route within our app.

Using nested routes in the app

In our app, we will use the **App** component for the app's shell, which renders the root path. We will then nest the products list page within this:

1. Open **App.tsx** and replace all the existing content with the following:

```

import { Outlet } from 'react-router-dom';

import { Header } from './Header';

export default function App() {

  return (

    <>

      <Header />

      <Outlet />

    </>
  );
}

```

```
);
```

```
}
```

The component renders the app header with nested content underneath it.

NOTE

*The empty JSX elements, `<>` and `</>`, are **React fragments**. React fragments are not added to the DOM and are used as a workaround to React components only being able to return a single element, so they are a way of returning multiple elements in a React component that keeps React happy.*

2. Open **Routes.tsx**, import the **App** component we just modified, and remove the **import** component for **Header**:

```
import {
```

```
  createBrowserRouter,
```

```
  RouterProvider,
```

```
} from 'react-router-dom';
```

```
import { ProductsPage } from '../pages/ProductsPage5';
```

```
import App from './App';
```

3. Update the **router** definition as follows:

```
const router = createBrowserRouter([
```

```
{
```

```
    path: '/',

    element: <App />,

    children: [

      {

        path: 'products',

        element: <ProductsPage />,

      }

    ]

  }

]);
```

The products list page is now nested inside the **App** component.

4. If you return to the running app, you will see that the app header now appears on the products list page. You will also see the **Products** link underlined because it is an active link:

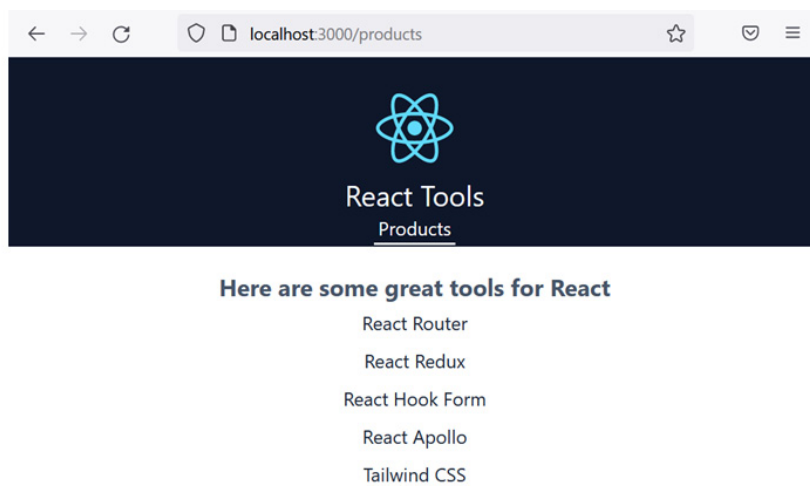


Figure 6.7 – App header on the products list page

To recap, nested routes allow components to be rendered for different path segments. An **Outlet** component is to render nested content within a parent component.

For more information on the **Outlet** component, see the following link: <https://reactrouter.com/en/main/components/outlet>.

Next, we will learn about route parameters.

Using route parameters

In this section, we will understand **route parameters** and how they are useful before using a route parameter in our app.

Understanding route parameters

A route parameter is a segment in the path that varies. The value of the variable segment is available to components so that they can render something conditionally.

In the following path, **1234** is the ID of a customer: **/customers/1234/**.

This can be defined as a route parameter in a route as follows:

```
{ path: '/customer/:id', element: <Customer /> }
```

A colon (:) followed by a name defines a route parameter. It is up to us to choose a parameter name that makes sense, so the **:id** segment in the path is the route parameter definition in the preceding route.

Multiple route parameters can be used in a path as follows:

```
{
```

```

    path: '/customer/:customerId/tasks/:taskId',
    element: <CustomerTask />,
  }

```

Route parameter names obviously have to be unique within a path.

Route parameters are available to components using React Router's **useParams** hook. The following code snippet is an example of how the **customerId** and **taskId** route parameter values could be obtained:

```

const params = useParams<Params>();
console.log('Customer id', params.customerId);
console.log('Task id', params.taskId);

```

As we can see from the code snippet, **useParams** has a generic argument that defines the type for the parameters. The **type** definition for the preceding code snippet is as follows:

```

type Params = {
  customerId: string;
  taskId: string;
};

```

It is important to note that the route parameter values are always strings because they are extracted from paths, which are strings.

Now that we understand route parameters, we will use a route parameter in our app.

Using route parameters in the app

We will add a product page to our app to show the description and price of each product. The path to the page will have a route parameter for the product ID. Carry out the following steps to implement the product page:

1. We will start by creating the product page. In the **src/pages** folder, create a file called **ProductPage.tsx** with the following **import** statements:

```

import { useParams } from 'react-router-dom';

import { products } from '../data/products';

```

We have imported the **useParams** hook from React Router, which will allow us to get the value of an **id** route parameter – the product's ID. We

have also imported the **products** array.

2. Start creating the **ProductPage** component as follows:

```
type Params = {

  id: string;

};

export function ProductPage() {

  const params = useParams<Params>();

  const id =

    params.id === undefined ? undefined :    parseInt(params.id);

}
```

We use the **useParams** hook to obtain the **id** route parameter and turn it into an integer if it has a value.

3. Now, add a variable that is assigned to the product with the ID from the route parameter:

```
export function ProductPage() {

  const params = useParams<Params>();

  const id =
```

```
params.id === undefined ? undefined : parseInt(params.id);
```

```
const product = products.find(
```

```
(product) => product.id === id
```

```
);
```

```
}
```

4. Return the product information from the **product** variable in the JSX:

```
export function ProductPage() {
```

```
...
```

```
return (
```

```
<div className="text-center p-5 text-xl">
```

```
{product === undefined ? (
```

```
<h1 className="text-xl text-slate-900">
```

```
Unknown product
```

```
</h1>
```

```
) : (
```

```
<>
```

```
<h1 className="text-xl text-slate-900">
```

```
{product.name}
```

```
</h1>
```

```
<p className="text-base text-slate-800">
```

```
{product.description}
```

```
</p>
```

```
<p className="text-base text-slate-800">
```

```
{new Intl.NumberFormat('en-US', {
```

```
  currency: 'USD',
```

```
  style: 'currency',
```

```
}).format(product.price)}
```

```
</p>
```

```
</>
```

```
)}  

```

```
</div>
```

```
);  

```

```
}  

```

Unknown product is returned if the product can't be found. Its name, description, and price are returned if the product is found. We use JavaScript's **Intl.NumberFormat** function to nicely format the price.

That completes the product page.

5. The next task is to add the route for the product page. Open **Routes.tsx** and add an **import** statement for the product page:

```
import { ProductPage } from './pages/ProductPage';
```

6. Add the following highlighted route for the product page:

```
const router = createBrowserRouter([
```

```
{
```

```
  path: '/',
```

```
  element: <App />,
```

```
children: [  
  
  {  
  
    path: 'products',  
  
    element: <ProductsPage />,  
  
  },  
  
  {  
  
    path: 'products/:id',  
  
    element: <ProductPage />,  
  
  }  
  
]  
  
}  
  
]);
```

So, the **/products/2** path should return a product page for React Redux.

7. In the running app, change the browser URL to <http://localhost:3000/products/2>. The React Redux product should show up:



Figure 6.8 – The product page

8. The last task in this section is to turn the products list on the products list page into links that open the relevant product page. Open **ProductsPage.tsx** and import the **Link** component from React Router:

```
import { Link } from 'react-router-dom';
```

9. Add a **Link** component around the product name in the JSX:

```
<ul className="list-none m-0 p-0">

  {products.map((product) => (

    <li key={product.id}>

      <Link

        to={`/${product.id}`}

        className="p-1 text-base text-slate-800"          hover:underline"

      >

        {product.name}

      </Link>

    </li>

  )}

</ul>
```

```
</Link>
```

```
</li>
```

```
))}
```

```
</ul>
```

Link paths are relative to the component's path. Given that the component path is `/products`, we set the link path to the product ID, which should match the `product` route.

10. Return to the running app and go to the products list page. Hover over the products and you will now see that they are links:

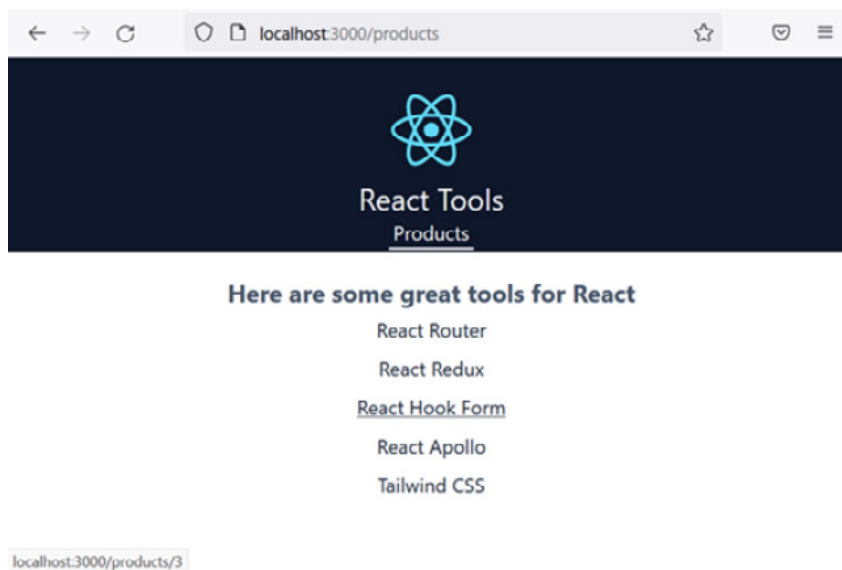


Figure 6.9 – Product list links

11. Click one of the products and the relevant product page will appear.

That completes this section on route parameters. Here's a quick recap:

- A route parameter is a varying segment in a path defined using a colon followed by the parameter name
- Route parameters can be accessed using React Router's `useParams` hook

For more information on the `useParams` hook, see the following link in the React Router documentation:

<https://reactrouter.com/en/main/hooks/use-params>.

Remember React Router’s error page we experienced in the *Declaring routes* section? Next, we will learn how to customize that error page.

Creating an error page

In this section, we will understand how error pages work in React Router before implementing one in our app.

Understanding error pages

Currently, a React Router built-in error page is shown when an error occurs. We can check this by entering an invalid path in the running app:

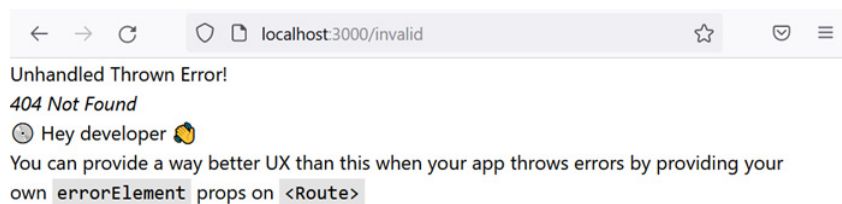


Figure 6.10 – Standard React Router error page

An error is raised because no matching routes are found in the router. The *404 Not Found* message on the error page confirms this.

This standard error page isn’t ideal because the information is targeted at a developer rather than a real user. Also, the app header isn’t shown, so users can’t easily navigate to a page that does exist.

As the error message suggests, an **errorElement** prop can be used on a route to override the standard error page. The following is an example of a custom error page defined for a customer’s route; if any error occurs on this route, the **CustomersErrorPage** component will be rendered:

```
const router = createBrowserRouter([
  ...,
  {
    path: 'customers',
    element: <CustomersPage />,
    errorElement: <CustomersErrorPage />
  },
  ...
]);
```

Now that we have started understanding error pages in React Router, we will implement one in our app.

Adding an error page

Carry out the steps below to create an error page in the app:

1. First, create a new page called **ErrorPage.tsx** in the **src/pages** folder with the following content:

```
import { Header } from '../Header';

export function ErrorPage() {

  return (

    <>

      <Header />

      <div className="text-center p-5 text-xl">

        <h1 className="text-xl text-slate-900">

          Sorry, an error has occurred

        </h1>

      </div>

    </>

  );

}
```

The component simply returns the app header with a *Sorry, an error has occurred* message underneath.

2. Open **Routes.tsx** and add an **import** statement for the error page:

```
import { ErrorPage } from './pages/ErrorPage';
```

3. Specify the error page on the root route as follows:

```
const router = createBrowserRouter([

  {

    path: '/',

    element: <App />,

    errorElement: <ErrorPage />,

    children: ...

  },

]);
```

Specifying the error page on the root route means that it will show if any routes have errors.

4. Switch back to the running app and change the browser URL to <http://localhost:3000/invalid>. The error page will be shown:

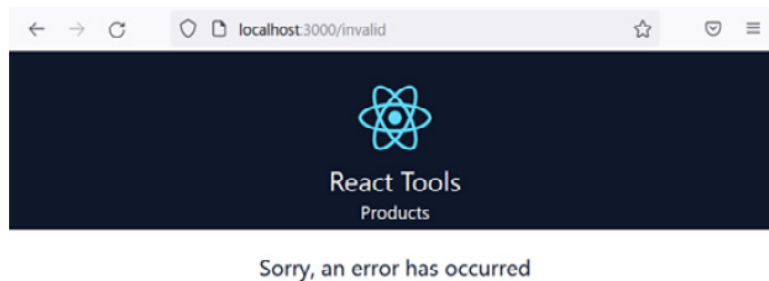


Figure 6.11 – Error page

5. This is a good start, but we can improve it by providing the user with more information, which we can get from React Router's `useRouteError` hook. Open `ErrorMessage.tsx` again and add an `import` statement for `useRouteError`:

```
import { useRouteError } from 'react-router-dom';
```

6. Assign the error to an `error` variable before the component's `return` statement using `useRouteError`:

```
export function ErrorMessage() {  
  
  const error = useRouteError();  
  
  return ...  
  
}
```

7. The `error` variable is of the `unknown` type – you can verify this by hovering over it. We can use a type predicate function to allow TypeScript to narrow it to something we can work with. Add the following type predicate function beneath the component:

```
function isError(error: any): error is { statusText: string } {  
  
  return "statusText" in error;  
  
}
```

The function checks whether the error object has a **statusText** property and if so, gives it a type with this property.

8. We can now use this function to render the information in the **status-Text** property:

```
return (  
  
  <>  
  
  <Header />  
  
  <div className="text-center p-5 text-xl">  
  
    <h1 className="text-xl text-slate-900">  
  
      Sorry, an error has occurred  
  
    </h1>  
  
    {isError(error) && (  
  
      <p className="text-base text-slate-700">  
  
        {error.statusText}  
  
      </p>  
  
    )}  
  )}
```

```
</div>
```

```
</>
```

```
);
```

9. In the running app, the information about the error appears on the error page as an invalid path:

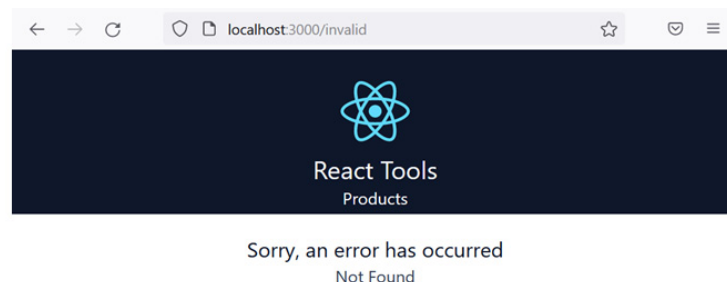


Figure 6.12 – Error page containing information about the error

That completes this section on error pages. The key point is to use an **errorElement** prop on a route to catch and display errors. Specific error information can be obtained using the **useRouteError** hook.

For more information on **errorElement**, see the following link:

<https://reactrouter.com/en/main/route/error-element>. For more information on the **useRouteError** hook, see the following link:

<https://reactrouter.com/en/main/hooks/use-route-error>.

Next, we will learn about index routes.

Using index routes

Currently, the app's root path displays nothing other than the header. In this section, we will learn about index routes to display a nice welcome message in the root path.

Understanding index routes

An **index route** can be thought of as a default child route. In React Router, if no children match a parent route, it will display an index route if one is defined. An index route has no path and instead has an **index** Boolean property, as in the following example:

```
{
  path: "/",
  element: <App />,
}
```

```

    children: [
      {
        index: true,
        element: <HomePage />,
      },
      ...,
    ]
  }
}

```

Next, we will add a home page using an index route in our app.

Using an index route in the app

Carry out the following steps to add a home page using an index route in our app:

1. Create a new file in the **src/pages** folder called **HomePage.tsx** with the following content:

```

export function HomePage() {

  return (

    <div className="text-center p-5 text-xl">

      <h1 className="text-xl text-slate-900">Welcome to      React Tools!</h1>

    </div>

  );

}

```

The page displays a welcome message.

2. Open **Routes.tsx** and import the home page we just created:

```

import { HomePage } from './pages/HomePage';

```

3. Add the home page as an index page for the root path as follows:

```
const router = createBrowserRouter([

  {

    path: '/',

    element: <App />,

    errorElement: <ErrorPage />,

    children: [

      {

        index: true,

        element: <HomePage />,

      },

      ...

    ]

  }

])
```

```
]);
```

4. We will add links to the logo and app title in the header to go to the home page. Open **Header.tsx** and import the **Link** component from React Router:

```
import { NavLink, Link } from 'react-router-dom';
```

5. Wrap links to the root page around the logo and title as follows:

```
<header ...>
```

```
  <Link to="/">
```

```
    <img src={logo} ... />
```

```
  </Link>
```

```
  <Link to="/">
```

```
    <h1 ...>React Tools</h1>
```

```
  </Link>
```

```
</nav>
```

```
...
```

```
</nav>
```

```
</header>
```

6. In the running app, click the app title to go to the root page, and you will see the welcome message displayed:

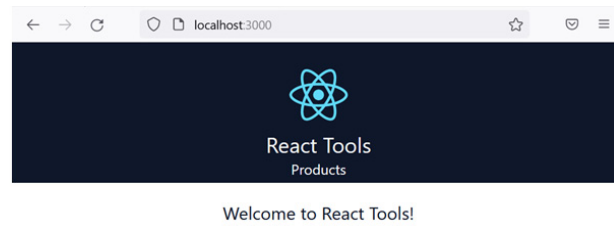


Figure 6.13 – Welcome page

That completes this section on index routes.

To recap, an index route is a default child route and is defined using an **index** Boolean property.

For more information on index routes, see the following link:

<https://reactrouter.com/en/main/route/route#index>.

Next, we will learn about search parameters.

Using search parameters

In this section, we will learn about search parameters in React Router and use them to implement a search feature in the app.

Understanding search parameters

Search parameters are part of a URL that comes after the **?** character and separated by the **&** character. Search parameters are sometimes referred to as **query parameters**. In the following URL, **type** and **when** are search parameters: **https://somewhere.com/?type=sometype&when=recent**.

React Router has a hook that returns functions for getting and setting search parameters called **useSearchParams**:

```
const [searchParams, setSearchParams] = useSearchParams();
```

searchParams is a JavaScript **URLSearchParams** object. There is a **get** method on **URLSearchParams**, which can be used to get the value of a search parameter. The following example gets the value of a search parameter called **type**:

```
const type = searchParams.get('type');
```

setSearchParams is a function used to set search parameter values. The function parameter is an object as in the following example:

```
setSearchParams({ type: 'sometype', when: 'recent' });
```

Next, we will add search functionality to our app.

Adding search functionality to the app

We will add a search box to the header of the app. Submitting a search will take the user to the products list page and list a filtered set of products matching the search criteria. Carry out the following steps:

1. Open **Header.tsx** and add **useSearchParams** to the React Router import. Also, add an **import** statement for the **FormEvent** type from React:

```
import { FormEvent } from 'react';
```

```
import {
```

```
  NavLink,
```

```
  Link,
```

```
  useSearchParams
```

```
} from 'react-router-dom';
```

2. Use the **useSearchParams** hook to destructure functions to get and set search parameters before the **return** statement:

```
export function Header() {
```

```
  const [searchParams, setSearchParams] = useSearchParams();
```

```
return ...
```

```
}
```

3. Add the following search form above the logo:

```
<header ...>
```

```
<form
```

```
  className="relative text-right"
```

```
  onSubmit={handleSearchSubmit}
```

```
>
```

```
<input
```

```
  type="search"
```

```
  name="search"
```

```
  placeholder="Search"
```

```
  defaultValue={searchParams.get('search') ?? ''}
```

```
  className="absolute right-0 top-0 rounded py-2 px-3 text-gray-700"
```

```
/>
```

```
</form>
```

```
<Link to="">
```

```
<img src={logo} ... />
```

```
</Link>
```

```
...
```

```
</header>
```

The form contains a search box with its default value as the value of a **search** parameter. `searchParams.get` returns **null** if the parameter doesn't exist, so a **nullish coalescing operator** (`??`) is used to set the search box's default value to an empty string in this case.

NOTE

*The **nullish coalescing operator** (`??`) returns the right operand if the left operand is **null** or **undefined**; otherwise, it returns the left operand. For more information, see the following link:*

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Nullish_coalescing.

The form submission calls a **handleSearchSubmit** function, which we'll implement in the next step.

4. Add a **handleSearchSubmit** function as follows, just above the **return** statement:

```
export function Header() {
```

```
  const [searchParams, setSearchParams] = useSearchParams();
```

```

function handleSearchSubmit(e:      FormEvent<HTMLFormElement>) {

    e.preventDefault();

    const formData = new FormData(e.currentTarget);

    const search = formData.get('search') as string;

    setSearchParams({ search });

}

return ...

}

```

The submit handler parameter is typed using **FormEvent**. **FormEvent** is a generic type that takes in the type of the element, which is **HTMLFormElement** for a form submit handler.

We use the **preventDefault** method on submit handler parameters to prevent the form from being submitted to the server because we handle all the logic in this function.

We use the JavaScript **FormData** interface to get the value of the search field. Then, we use a type assertion to set the type of the search field value to a string.

The last line of code in the submit handler sets the value of the search parameter. This will update the browser's URL to have this search parameter.

NOTE

We will learn a lot more about forms in React later , Working with Forms.

5. Now, we need to filter the products list with the value of the search parameter. Open **ProductsPage.tsx** and add **useSearchParams** to the **import** statement:

```
import { Link, useSearchParams } from 'react-router-dom';
```

6. At the top of the **ProductsPage** component, destructure **searchParams** from **useSearchParams** as follows:

```
export function ProductsPage() {  
  
  const [searchParams] = useSearchParams();  
  
  return ...  
  
}
```

7. Add the following function just before the **return** statement to filter the products list by the search value:

```
const [searchParams] = useSearchParams();  
  
function getFilteredProducts() {  
  
  const search = searchParams.get('search');  
  
  if (search === null || search === "") {  
  
    return products;  
  
  } else {
```

```

    return products.filter(

      (product) =>

        product.name

          .toLowerCase()

            .indexOf(search.toLowerCase()) > -1

    );

  }

}

return ...

```

The function starts by getting the value of the **search** parameter. The full product list is returned if there is no search parameter or if the value is an empty string. Otherwise, the product list is filtered using the array's **filter** function, checking that the search value is contained within the product name irrespective of the case.

8. Use the function we just created in the JSX to output the filtered products. Replace the reference to **products** with a call to **getFilteredProducts** as follows:

```

<ul className="list-none m-0 p-0">

  {getFilteredProducts().map((product) => (

```

```
<li
```

```
  key={product.id}
```

```
  className="p-1 text-base text-slate-800"
```

```
>
```

```
  <Link
```

```
    to={`/${product.id}`}
```

```
    className="p-1 text-base text-slate-800"
```

```
    hover:underline"
```

```
>
```

```
    {product.name}
```

```
  </Link>
```

```
</li>
```

```
  )})
```

```
</ul>
```

9. In the running app, whilst on the home page, enter some search criteria in the search box and press *Enter* to submit the search.

The search parameter is added to the URL in the browser. However, it doesn't navigate to the products list page. Don't worry about this because we'll address this issue in the next section:

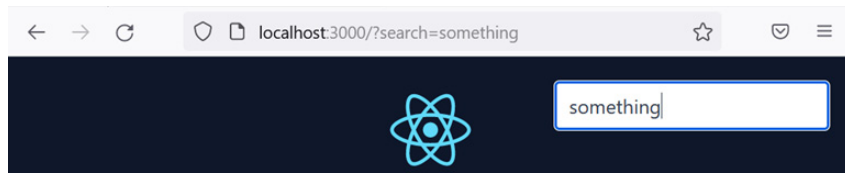


Figure 6.14 – The search parameter added to the URL

The key point in this section is that the `useSearchParams` hook from React Router allows you to set and get URL search parameters. The parameters are also structured in a JavaScript `URLSearchParams` object.

For more information on the `useSearchParams` hook, see the following link in the React Router documentation:

<https://reactrouter.com/en/main/hooks/use-search-params>. More information on `URLSearchParams` is available at

<https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams>.

Next, we will explore another React Router hook that enables programmatic navigation.

Navigating programmatically

React Router's **Link** and **NavLink** components allow declarative navigation. However, sometimes we must navigate imperatively – in fact, this would be useful for the search feature in our app to navigate to the products list page. In this section, we will learn how to programmatically navigate with React Router and use this to complete the app's search feature. Carry out the following steps:

1. Open **Header.tsx** and add import the **useNavigate** hook from React Router:

```
import {  
  
  NavLink,  
  
  Link,  
  
  useSearchParams,
```

useNavigate

```
} from 'react-router-dom';
```

The **useNavigate** hook returns a function we can use to perform programmatic navigation.

2. Invoke **useNavigate** after the call to the **useSearchParams** hook. Assign the result to a variable called **navigate**:

```
export function Header() {  
  
    const [searchParams, setSearchParams] = useSearchParams();  
  
    const navigate = useNavigate();  
  
    ...  
  
}
```

The **navigate** variable is a function that can be used to navigate. It takes in an argument for the path to navigate to.

3. In **handleSearchSubmit**, replace the **setSearchParams** call with a call to **navigate** to go to the products list page with the relevant search parameter:

```
function handleSearchSubmit(e: FormEvent<HTMLFormElement>) {  
  
    e.preventDefault();  
  
    const formData = new FormData(e.currentTarget);
```

```
const search = formData.get('search') as string;
```

```
navigate(`/products/?search=${search}`);
```

```
}
```

4. We no longer need **setSearchParams** because the setting of the search parameter is included in the navigation path, so remove this from the **useSearchParams** call:

```
const [searchParams] = useSearchParams();
```

5. In the running app, enter some search criteria in the search box and press *Enter* to submit the search.

The search parameter is used to navigate to the products list page. When the products list page appears, the correctly filtered products are shown:

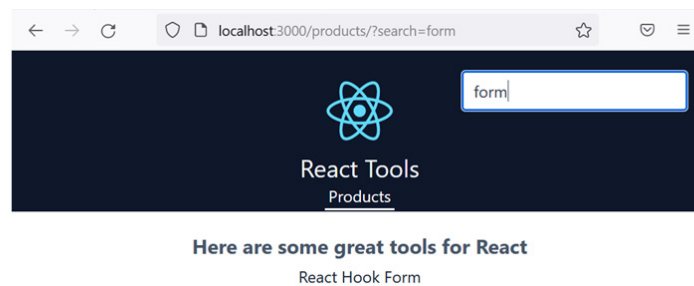


Figure 6.15 – Products list page with filter products

So, programmatic navigation is achieved using the **useNavigate** hook. This returns a function that can navigate to the path passed into it.

For more information on the **useNavigate** hook, see the following link in the React Router documentation:

<https://reactrouter.com/en/main/hooks/use-navigate>.

Next, we will refactor the search form's navigation to use React Router's **Form** component.

Using form navigation

In this section, we will use React Router's **Form** component to navigate to the products list page when the search criteria are submitted. **Form** is a wrapper around the HTML **form** element that handles the form submission on the client side. This will replace the use of **useNavigate** and simplify the code. Carry out the following steps:

1. In **Header.tsx**, start by removing **useNavigate** from **import** for the React Router and replace it with the **Form** component:

```
import {  
  
  NavLink,  
  
  Link,  
  
  useSearchParams,  
  
  Form  
  
} from 'react-router-dom';
```

2. In the JSX, replace the **form** element with React Router's **Form** component:

```
<Form  
  
  className="relative text-right"  
  
  onSubmit={handleSearchSubmit}  
  
>  
  
  <input ... />  
  
</Form>
```

3. In the **Form** element in the JSX, remove the **onSubmit** handler. Replace this with the following **action** attribute so that the form is sent to the **products** route:

```
<Form  
  
  className="relative text-right"  
  
  action="/products"  
  
>  
  
  ...  
  
</Form>
```

React Router's form submission mimics how a native **form** element submits to a server path. However, React Router submits the form to a client-side route instead. In addition, **Form** mimics an HTTP **GET** request by default, so a **search** parameter will automatically be added to the URL.

4. The remaining tasks are to remove the following code:
 1. Remove the React import statement because **FormEvent** is redundant now
 2. Remove the call to **useNavigate** because this is no longer required
 3. Remove the **handleSearchSubmit** function because this is no longer required
5. In the running app, enter some search criteria in the search box and press *Enter* to submit the search. This will work as it did before.

That has simplified the code quite a bit!

We will learn more about React Router's **Form** component later . The key takeaway from this section is that **Form** wraps the HTML **form** element, handling form submission on the client.

For more information on **Form**, see the following link in the React Router documentation: <https://reactrouter.com/en/main/components/form>.

Next, we will learn about a type of performance optimization that can be applied to large pages in the app.

Implementing lazy loading

Currently, all the JavaScript for our app is loaded when the app first loads. This can be problematic in large apps. In this section, we will learn how to only load the JavaScript for components when their route becomes active. This pattern is often referred to as **lazy loading**. In our app, we will create a lazily loaded admin page.

Understanding React lazy loading

By default, all React components are bundled together and loaded when the app first loads. This is inefficient for large apps – particularly when a user does not use many components. Lazily loading React components addresses this issue because lazy components aren't included in the initial bundle that is loaded; instead, their JavaScript is fetched and loaded when rendered.

There are two main steps to lazy loading React components. First, the component must be dynamically imported as follows:

```
const LazyPage = lazy(() => import('./LazyPage'));
```

In the code block, **lazy** is a function from React that enables the imported component to be lazily loaded. Note that the lazy page must be a default export – lazy loading doesn't work with named exports.

Webpack can then split the JavaScript for **LazyPage** into a separate bundle. Note that this separate bundle will include any child components of **LazyPage**.

The second step is to render the lazy component inside React's **Suspense** component as follows:

```
<Route
  path="lazy"
  element={
    <Suspense fallback={<div>Loading...</div>}>
      <LazyPage />
    </Suspense>
  }
/>
```

The **Suspense** component's **fallback** prop can be set to an element to render while the lazy page is being fetched.

Next, we will create a lazy admin page in our app.

Adding a lazy admin page to the app

Carry out the following steps to add a lazy admin page to our app:

1. Create a file called **AdminPage.tsx** in the **src/pages** folder with the following content:

```
export default function AdminPage() {

  return (

    <div className="text-center p-5 text-xl">

      <h1 className="text-xl text-slate-00">Admin Panel</h1>

      <p className="text-base text-slate-00">

        You shou'dn't come here often becaus' I'm lazy

      </p>

    </div>

  );

}
```

The page is very small, so it is not a great use case for lazy loading. However, its simplicity will allow us to focus on how to implement lazy loading.

2. Open **Routes.tsx** and import **lazy** and **Suspense** from React:

```
import { lazy, Suspense } from 'react';
```

3. Import the admin page as follows (it is important that this comes after all the other **import** statements):

```
const AdminPage = lazy(() => import('./pages/AdminPage'));
```

4. Add the **admin** route as follows:

```
const router = createBrowserRouter([
```

```
{
```

```
  path: '/',
```

```
  element: <App />,
```

```
  errorElement: <ErrorPage />,
```

```
  children: [
```

```
    ...,
```

```
    {
```

```
      path: 'admin',
```

```
      element: (
```

```
        <Suspense
```

```

        fallback={

            <div className="text-center p-5 text-xl text-slate-00

                Loading...

            </div>

        }

    >

    <AdminPage />

</Suspense>

)

}

]

}

]);

```

The path to the admin page is `/admin`. A loading indicator will render as the admin page's JavaScript is fetched.

5. Open **Header.tsx** and add a link to the admin page after the **Products** link as follows:

```
<nav>
```

```
<NavLink ... >
```

```
Products
```

```
</NavLink>
```

```
<NavLink
```

```
"o="ad"in"
```

```
className={({ isActive }) =>
```

```
`text-white no-underline p-1 pb-0.5 border-solid border-b-2 ${
```

```
isActive? "border-white": "border-transparent"
```

```
}`
```

```
}
```

```
>
```

```
Admin
```

</NavLink>

</nav>

6. In the running app, open the browser DevTools and go to the **Network** tab and clear out any existing requests. Slow down the connection by selecting **Slow 3G** from the **No throttling** menu:

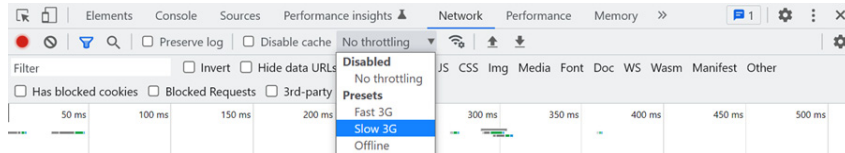


Figure 6.16 – Setting a slow connection

7. Now, click on the **Admin** link in the header. The loading indicator appears because the JavaScript for the admin page is being downloaded:

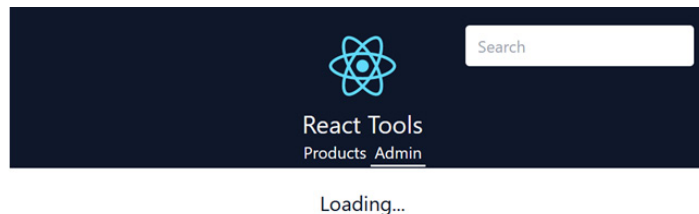


Figure 6.17 – The loading indicator

After the admin page has been downloaded, it will render in the browser. If you look at the **Network** tab in DevTools, you will see confirmation of the admin page bundle being lazily loaded:

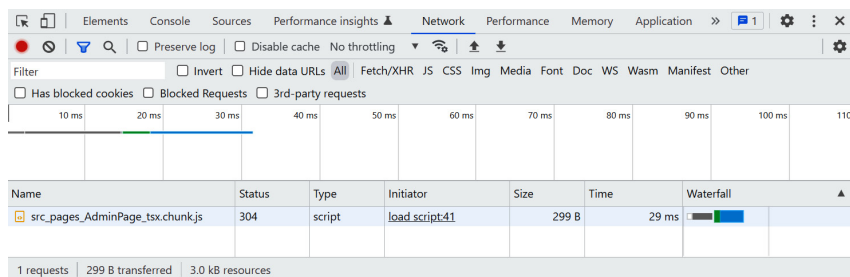


Figure 6.18 – Admin page download

That completes this section on lazily loading React components. In summary, lazily loading React components is achieved by dynamically importing the component file and rendering the component inside a **Suspense** component. That also completes this part. Next, we will recap what we have learned about React Router.

Summary

React Router gives us a comprehensive set of components and hooks for managing the navigation between pages in our app. We used **createBrowserRouter** to define all our web app's routes. A route contains a path and a component to render when the path matches the browser URL. We used an **errorElement** prop for a route to render a custom error page in our app.

We used nested routes to allow the **App** component to render the app shell and page components within it. We used React Router's **Outlet** component inside the **App** component to render page content. We also used an index route on the root route to render a welcome message.

We used React Router's **NavLink** component to render navigation links that are highlighted when their route is active. The **Link** component is great for other links that have static styling requirements – we used this for product links on the product list. We used React Router's **Form** component to navigate to the products list page when the search form is submitted.

Route parameters and search parameters allow parameters to be passed into components so that they can render dynamic content. **useParams** gives access to route parameters, and **useSearchParams** provides access to search parameters.

React components can be lazily loaded to increase startup performance. This is achieved by dynamically importing the component file and rendering the component inside a **Suspense** component.

In the next part, we will learn all about forms in React.

Questions

Let's test our knowledge of React Router with the following questions:

1. We have declared the following routes in an app:

```
const router = createBrowserRouter([

  {

    path: "/",
```

```

    element: <App />,

    errorElement: <ErrorPage />,

    children: [

        { path: "customers", element: <CustomersPage /> }

    ]

}

]);

```

What component will render when the path is **/customers**?

What component will render when the path is **/products**?

2. What would the path be in a route that could handle a **/customers/37** path? **37** is a customer ID and could vary.
3. The routes for a **settings** page are defined as follows:

```

{

    path: "/settings",

    element: <SettingsPage />,

    children: [

        { path: "general", element: <GeneralSettingsTab /> },

```

```

    { path: "dangerous", element: <DangerousSettingsTab /> }

  ]

}

```

The settings page has **General** and **Dangerous** tabs, which show when the path is `/settings/general` and `/settings/dangerous`, respectively. However, when these paths are requested, no tab content is shown on the settings page – so, what could we have forgotten to add in the **SettingsPage** component?

4. We are implementing a navigation bar in an app. When clicking on a navigation item, the app should navigate to the relevant page. Which React Router component should we use to render the navigational items? **Link** or **NavLink**?
5. A route is defined as follows:

```
{ path: "/user/:userId", element: <UserPage /> }
```

Inside the **UserPage** component, the following code is used to get the user **id** information from the browser URL:

```
const params = useParams<{id: string}>();
const id = params.id;
```

However, **id** is always **undefined**. What is the problem?

6. The following URL contains an example of a search parameter on a **customers** page:

`/customers/?search=cool company`

However, an error occurs in the following implementation:

```
function getFilteredCustomers() {
  const criteria = useSearchParams().get('search');
  if (criteria === null || criteria === "") {
    return customers;
  } else {
    return customers.filter(
      (customer) =>
        customer.name.toLowerCase().indexOf(criteria.
          toLowerCase()) > -1
    );
  }
}
```

```
    );  
  }  
}
```

What is the problem?

7. A React component is lazily loaded as follows:

```
const SpecialPage = lazy(() => import('./pages/SpecialPage'));
```

```
const router = createBrowserRouter([
```

```
  ...,
```

```
  {
```

```
    path: '/special',
```

```
    element: <SpecialPage />,
```

```
  },
```

```
  ...
```

```
]);
```

However, React throws an error. What is the problem?

Answers

1. **CustomersPage** will render when the path is **/customers**.

ErrorPage will render when the path is **/products**.

2. The path could be **path="customers/:customerId"**.

3. It is likely that the **Outlet** component has not been added to **SettingsPage**.
4. Both will work, but **NavLink** is better because it enables items to be styled when active.
5. The route parameter referenced should be **userId**:

```
const params = useParams<{userId: string}>();
```

```
const id = params.userId;
```

6. Hooks must be called at the top level of function components. Also, the **useSearchParams** hook doesn't directly have a **get** method. Here's the corrected code:

```
const [searchParams] = useSearchParams();
```

```
function getFilteredCustomers() {
```

```
    const criteria = searchParams.get('search');
```

```
    ...
```

```
}
```

7. The lazy component must be nested inside a **Suspense** component as follows:

```
{
```

```
    path: '/special',
```

```
    element: (
```

```
        <Suspense fallback={<Loading />}>
```

<SpecialPage />

</Suspense>

)

}